[2026] Hướng dẫn dùng docker để nộp bài cho Zalo AI Challenge 2026 - LifeMate - Agentic AI That Turns Your Voice Into Action

1. **Giới thiệu**
2. Cài **đặt Docker trên** Ubuntu
3. Cài **đặt Nvidia Docker / Driver**
4. **Nộp bài cho Zalo AI Challange**
5. Cấu trúc thư mục code
6. **Bổ sung bắt buộc về Training Code & Tài liệu mô tả**
7. Phụ lục: Cấu trúc viết Jupyter notebook để đo lường thời gian
8. Upload docker

1. Giới thiệu

Tài liệu này hướng dẫn các đội thi Zalo AI Challenge 2026 đóng gói và nộp giải pháp dưới dạng Docker Image, bao gồm đầy đủ **mã nguồn inference, mô hình, mã nguồn** và **dữ liệu huấn luyện (Training Code & Data)**, cùng các **tài liệu** phục vụ kiểm chứng kết quả.

Docker Image phải được hoàn thiện và nộp trước khi công bố bộ private test. Quá trình inference phải hoạt động hoàn toàn offline để BTC có thể chạy lại, kiểm chứng và xác định kết quả cuối cùng.

Docker là một nền tảng giúp người dùng đóng gói và chạy chương trình của mình trên các môi trường khác nhau một cách nhanh nhất dựa trên các *container*.

Docker Image là một dạng tập hợp các tệp của ứng dụng, được tạo ra bởi Docker engine. Nội dung của các Docker image sẽ không bị thay đổi khi di chuyển. Docker image được dùng để chạy các Docker container.

Docker Container là một dạng runtime của các Docker image, dùng để làm môi trường chạy ứng dụng.

Hướng dẫn chi tiết các bạn có thể tham khảo tại: https://docs.docker.com/get-started/

2. Cài đặt Docker trên Ubuntu

Đối với các hệ điều hành khác, tham khảo cách cài đặt tại đây https://docs.docker.com/install/overview/

1. Cài đặt docker

   > sudo apt-get install docker.io

2. Kiểm tra phiên bản docker

   > sudo docker --version

   ⚠️ Docker version 24.0.5, build ced0996

3. Chạy thử docker hello world

   > sudo docker run hello-world

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

4. Một số câu lệnh phổ biến
    - Liệt kê các images hiện có

      > sudo docker images

    - Liệt kê các container hiện có

      > sudo docker ps -a



3. Cài đặt Nvidia Docker / Driver

Để sử dụng đươc GPU trên docker, bạn cần cài đặt Nvidia docker.

Hướng dẫn cài đặt Nvidia docker tại đây: https://github.com/NVIDIA/nvidia-docker

Lưu ý: Để tránh lỗi khi chấm, các đội cần cài đặt các thư viện **đồng bộ môi trường với server BTC**:

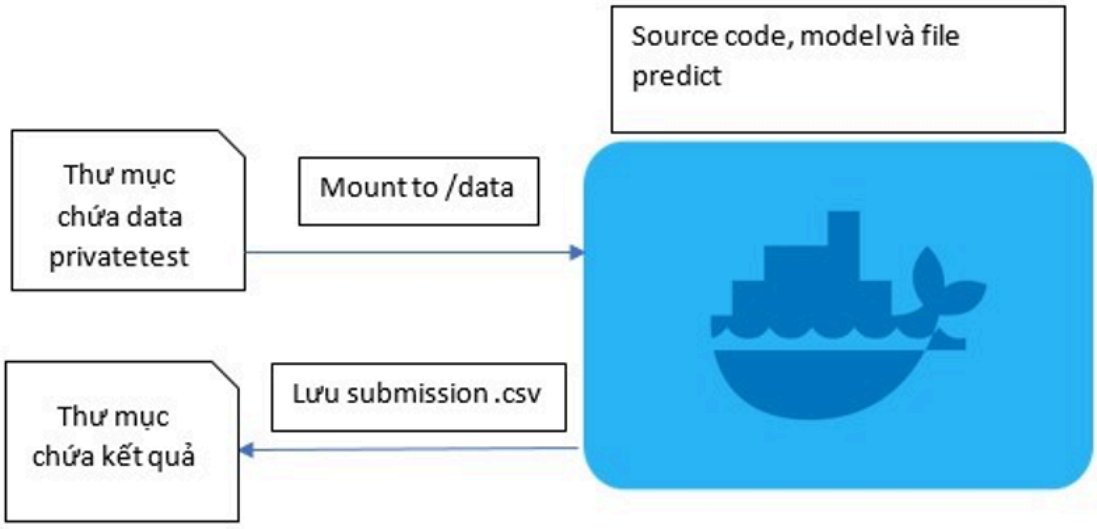
- **GPU: 1 × NVIDIA GeForce RTX 5090 (32 GB)**, CPU 16 nhân Intel(R) Xeon(R) Gold 6459C, RAM 64 GB
- **Driver Version: 595.58.03**
- **CUDA Version: 13.2.0**

Base image phải dùng CUDA **không cao hơn 13.2** (driver của BTC chạy được các image CUDA 12.x và 13.x ≤ 13.2). Ví dụ nvidia docker image: `nvidia/cuda:13.2.0-devel-ubuntu22.04.`

4. Nộp bài cho Zalo AI Challange

Các bước cần thiết để tạo được một Docker Image

1. Tạo một docker container mới (hoặc sử dụng docker có sẵn).
2. Đưa model và source code đang có vào docker.
3. Install các packages và libraries cần thiết mà solution của bạn sử dụng để chạy.
4. Viết một file script tên predict.sh. File này chứa command các bước để chạy test. Nhận input từ **/data** và output ra **/result/submission.jsonl** (tuỳ theo format quy định của đề bài). Ngoài ra, xuất ra các thông số thời gian load model, thời gian predict ở stdout.
5. Nộp file predict **predict_notebook.ipynb** để đo thời gian inference (xem phụ lục)
6. Commit các thay đổi trong docker container
7. Save docker container thành 1 file image kèm checksum và nộp lên website cuộc thi.



5. Cấu trúc thư mục code

Ví dụ source code của bạn ở folder **/mnt/zac2026/** với cấu trúc như sau.

```
|---- predict.py
|---- preprocessing.py
|---- saved_models
|      |---- models.safetensors # model cần đc copy vào trong docker
|---- requirements.txt
|---- predict.sh
|---- start_jupyter.sh
|---- predict_notebook.ipynb # sử dụng để đánh giá thời gian, xem hướng dẫn viết notebook ở cuối
|---- training_code # chứa toàn bộ mã nguồn training và README.
|      |---- README.md
```

Dữ liệu input (`/data`). BTC mount thư mục chứa split private test vào **/data**, cùng cấu trúc với gói public test

```
📂 /data/
    └─ private_test/
        ├---- private_test.jsonl                    # input từng mẫu: id, audio, context, history
        ├---- private_test_tools_catalog.json       # schema các tool
        ├---- private_test_environment/<id>/state.json  # KHÔNG được đọc trực tiếp, chỉ qua zaic_tools
        ├---- audio/<id>/<id>.wav                   # yêu cầu cuối; <id>_u1.wav, <id>_u2.wav...: các lượt user trong history
        ├---- zaic_tools/                           # thư viện chạy tool của private test
```

Sample `predict.py` tối thiểu:

```python
import json, sys, time
from pathlib import Path

ROOT = Path("/data")                                    # thư mục chứa split
SPLIT = "private_test"
sys.path.insert(0, str(ROOT / SPLIT))                   # zaic_tools của split, KHÔNG dùng bản đóng trong image
from zaic_tools import ToolEnv

def infer(item):
        t1 = time.perf_counter()                        # bắt đầu mẫu: trước khi đọc audio
    env = ToolEnv(SPLIT, item["id"], root=ROOT)     # đúng một ToolEnv cho mỗi mẫu
    # ASR (ROOT / SPLIT / item["audio"]) -> LLM -> env.call(tool, arguments) -> LLM -> ...
    return env.get_tool_calls(), env.get_state_hash(), int((t
```

```
ime.perf_counter() - t1) * 1000)

Path("/result").mkdir(parents=True, exist_ok=True)
with open(ROOT / SPLIT / f"{SPLIT}.jsonl", encoding="utf-8")
as fin, \
     open("/result/submission.jsonl", "w", encoding="utf-8")
as fout:
    for line in fin:
        item = json.loads(line)
        tool_calls, state_hash, ms = infer(item)
        fout.write(json.dumps({"id": item["id"], "tool_call
s": tool_calls, "state_hash": state_hash, "time": ms},
                              ensure_ascii=False) + "\n")
```

1. Khởi động docker container và đặt tên container với name **zac2026**

> **docker run --gpus '"device=0"' --network host -it --name zac2026 nvidia/cuda:13.2.0-devel-ubuntu22.04 /bin/bash**

- Bạn **bắt buộc** phải sử dụng base image phải có CUDA 13.2 để tương tích với server chấm điểm của ban tổ chức.
- Cờ **--network host** chỉ dùng lúc dựng image (cài package, chạy jupyter đo thời gian ở bước sau). Khi chấm, container **không có mạng**.

```
$ docker run --gpus '"device=0"' --network host  --hostname zac2026 -it --name zac2026 harbor.infra.zalo.services/nvcr_proxy_cache/nvid
ia/cuda:13.2.0-devel-ubuntu22.04 /bin/bash

==========
== CUDA ==
==========

CUDA Version 13.2.0

Container image Copyright (c) 2016-2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

A copy of this license is made available in this container at /NGC-DL-CONTAINER-LICENSE for your convenience.

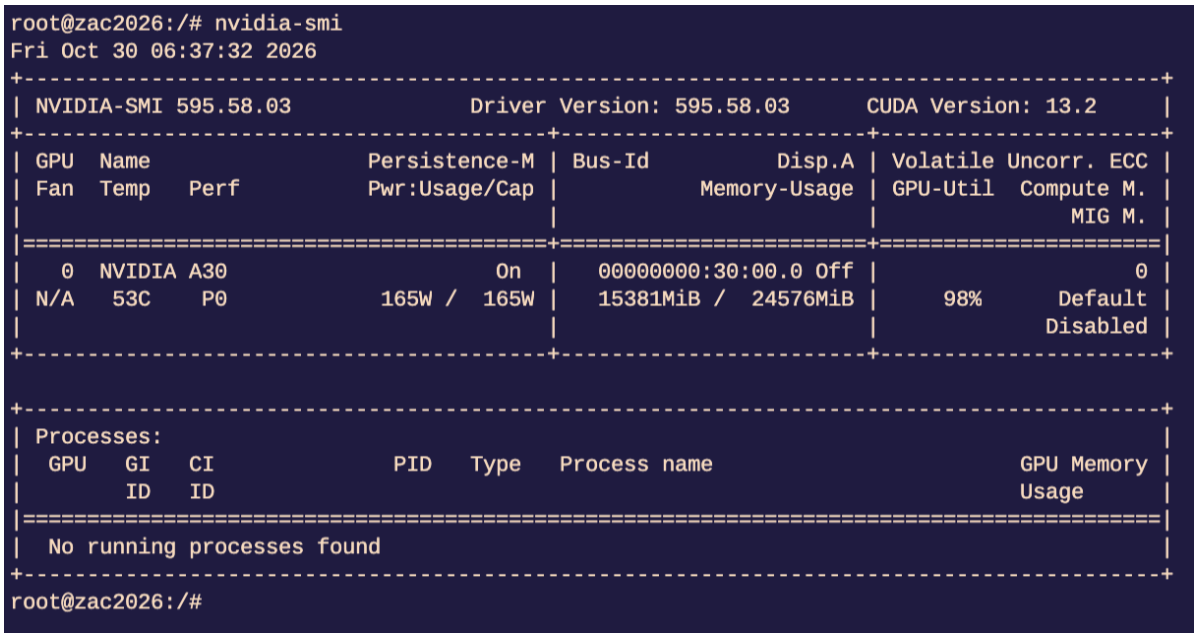
root@zac2026:/#
```

Lúc này trong container đang ở vị trí /

```
root@zac2026:/# pwd
/
```

2. Kiểm tra container có sử dụng được GPU hay không bằng lệnh nvidia-smi

> nvidia-smi



3. **Mở một terminal mới**, copy source code ở folder hiện tại vào trong container với cú pháp.

   sudo docker cp [source_path] [container_name]:[destination_path]

   > sudo docker cp /mnt/zac2026/ zac2026:/code/

   Lúc này Lúc này toàn bộ source code ở ngoài đã được copy vào trong container ở vị trí /code trong **container**. Mọi model (ASR, LLM, tokenizer...) phải nằm sẵn trong image, không tải từ HuggingFace lúc chạy.



4. Cài python và các package cần thiết

   > apt update; apt-get -y install libgl1-mesa-glx libglib2.0-0 vim ffmpeg; apt -y install python3-pip

   Nhập câu trả lời cần thiết nếu được hỏi

```
  1. Africa   3. Antarctica  5. Arctic  7. Atlantic  9. Indian    11. SystemV  13. Etc
  2. America  4. Australia   6. Asia    8. Europe    10. Pacific  12. US
Geographic area: 6

Please select the city or region corresponding to your time zone.

  1. Aden      16. Brunei       31. Hong_Kong    46. Kuala_Lumpur  61. Pyongyang      76. Tehran
  2. Almaty    17. Chita        32. Hovd         47. Kuching       62. Qatar          77. Tel_Aviv
  3. Amman     18. Choibalsan   33. Irkutsk      48. Kuwait        63. Qostanay       78. Thimphu
  4. Anadyr    19. Chongqing    34. Istanbul     49. Macau         64. Qyzylorda      79. Tokyo
  5. Aqtau     20. Colombo      35. Jakarta      50. Magadan       65. Rangoon        80. Tomsk
  6. Aqtobe    21. Damascus     36. Jayapura     51. Makassar      66. Riyadh         81. Ujung_Pandang
  7. Ashgabat  22. Dhaka        37. Jerusalem    52. Manila        67. Sakhalin       82. Ulaanbaatar
  8. Atyrau    23. Dili         38. Kabul        53. Muscat        68. Samarkand      83. Urumqi
  9. Baghdad   24. Dubai        39. Kamchatka    54. Nicosia       69. Seoul          84. Ust-Nera
  10. Bahrain  25. Dushanbe     40. Karachi      55. Novokuznetsk  70. Shanghai       85. Vientiane
  11. Baku     26. Famagusta    41. Kashgar      56. Novosibirsk   71. Singapore      86. Vladivostok
  12. Bangkok  27. Gaza         42. Kathmandu    57. Omsk          72. Srednekolymsk  87. Yakutsk
  13. Barnaul  28. Harbin       43. Khandyga     58. Oral          73. Taipei         88. Yangon
  14. Beirut   29. Hebron       44. Kolkata      59. Phnom_Penh    74. Tashkent       89. Yekaterinburg
  15. Bishkek  30. Ho_Chi_Minh  45. Krasnoyarsk  60. Pontianak     75. Tbilisi        90. Yerevan
Time zone: 30


Current default time zone: 'Asia/Ho_Chi_Minh'
Local time is now:      Fri Nov 24 14:52:01 +07 2023.
Universal Time is now:  Fri Nov 24 07:52:01 UTC 2023.
```

```
Hit:1 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64  InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
108 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libglib2.0-0 is already the newest version (2.72.4-0ubuntu2.10).
vim is already the newest version (2:8.2.3995-1ubuntu2.36).
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
libgl1-mesa-glx is already the newest version (23.0.4-0ubuntu1~22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 108 not upgraded.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (22.0.2+dfsg-1ubuntu0.7).
0 upgraded, 0 newly installed, 0 to remove and 108 not upgraded.
root@zac2026:/# which python3
/usr/bin/python3
root@zac2026:/# python3 -V
Python 3.10.12
root@zac2026:/#
```

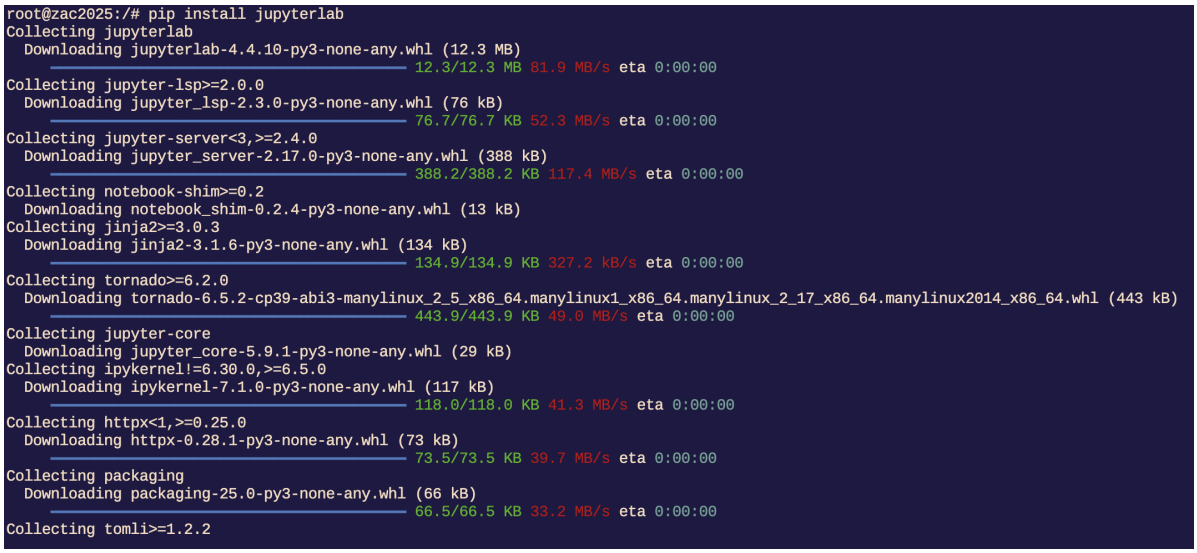
5. Cài đặt các thư viện cần thiết từ file requirements.txt hoặc tự install **trong container**

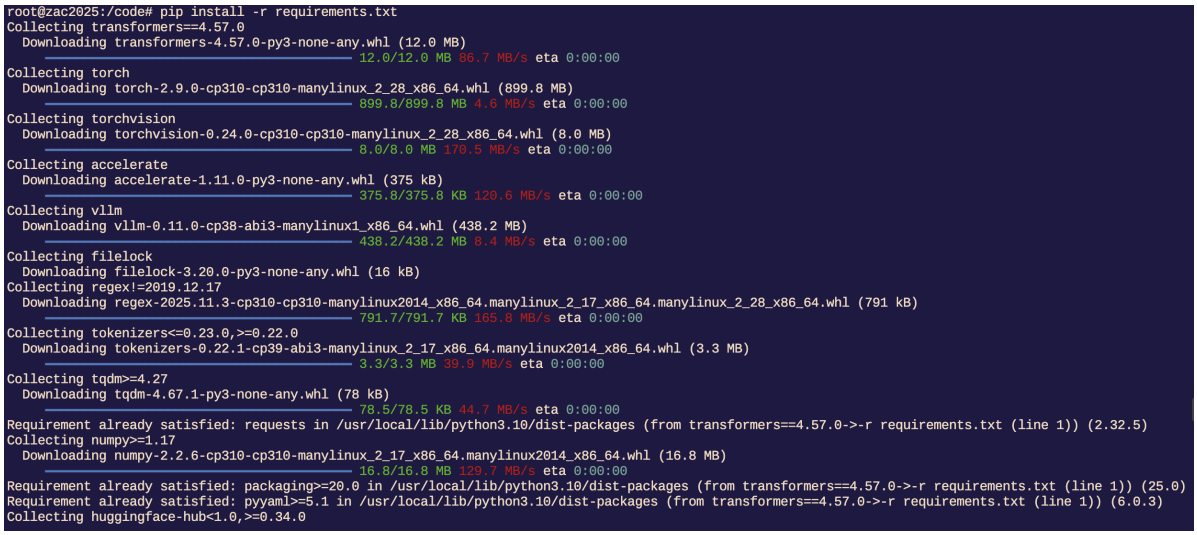
   Trước hết chuyển tới thư mục code bằng lệnh

   > cd /code

   > pip install jupyterlab
   > pip install -r requirements.txt

   Lưu ý môi trường bắt buộc phải cài đặt **jupyterlab** để chấm thời gian inference





6. Chỉnh sửa file **/code/predict.sh**

   Ví dụ có 2 bước để chạy bộ test ra kết quả.

   python3 /code/preprocessing.py
   python3 /code/predict.py

   File predict.py phải ghi kết quả ở **/result/submission.jsonl (trong container, code phải tự tạo folder /result nếu folder chưa tồn tại)**

   > bash /code/predict.sh

```
root@zac2026:/code# bash predict.sh
preprocessing...
output will be saved in /result/submission.jsonl
```

7. Chỉnh sửa file /code/start_jupyter.sh

```
jupyter lab --port 9777 --ip 0.0.0.0 --NotebookApp.passwor
d='zac2026' --NotebookApp.token='zac2026'  --allow-root --
no-browser
```

8. Chỉnh sửa file **predict_notebook.ipynb: Xem phụ lục ở cuối**
9. Sau khi hoàn thành predict tiến hành lưu lại các thay đổi trên container. (**thao tác ở terminal**)

   sudo docker commit [CONTAINER ID] [image name]:[tag name]

   > sudo docker commit zac2026 zac2026-teamname:v1

```
$ docker commit zac2026 zac2026-btc-test-docker:v1
sha256:b37ff9977f681c04a5976041c647d9716fe6ce49842edec2be20d2c00f67ba7b
```

10. Kiểm tra docker lần cuối

    10.1 Kiểm tra hàm predict để xuất submission.jsonl

    Chạy predict.sh (các tham số về hyperparameters phải phải được set default trong file **predict.py**, BTC không chịu trách nhiệm nếu các bạn muốn thay đổi tham số này sau khi nộp bài)

    sudo docker run –v [path to test data]:/data –v [current dir]:/result [docker name]

    > sudo docker run --gpus '"device=0"' -v /mnt/zac2026/data:/data:ro -v /mnt/zac2026/result:/result zac2026-btc-test-docker:v1 /bin/bash /code/predict.sh

```
$ docker run --gpus '"device=0"'  -v /mnt/zac2026/data:/data:ro -v /mnt/zac2026/result:/result zac2026-btc-test-docker:v1 /bin/bash /co
de/predict.sh

==========
== CUDA ==
==========

CUDA Version 13.2.0

Container image Copyright (c) 2016-2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

A copy of this license is made available in this container at /NGC-DL-CONTAINER-LICENSE for your convenience.

preprocessing...
preprocessing: 634 mẫu, thiếu audio: 0
output will be saved in /result/submission.jsonl
load model: 2 ms
predict: 634 mẫu, 1128 ms, trung bình 1.8 ms/mẫu
```

```
$ pwd && ls result/
/mnt/zac2026
submission.jsonl
```

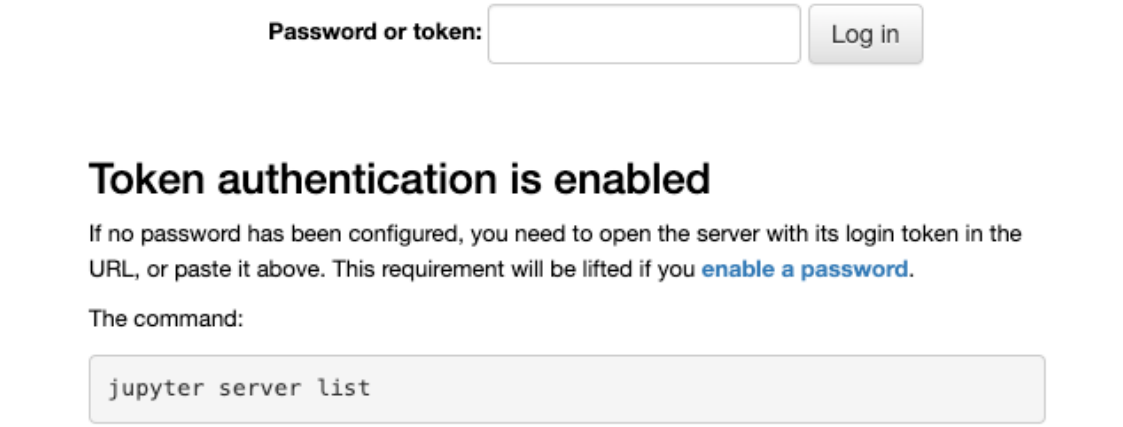
Kết quả của file submission.jsonl sẽ được trả về ở thư mục **/mnt/zac2026/result**

10.2 Kiểm tra jupyter có chạy được không.

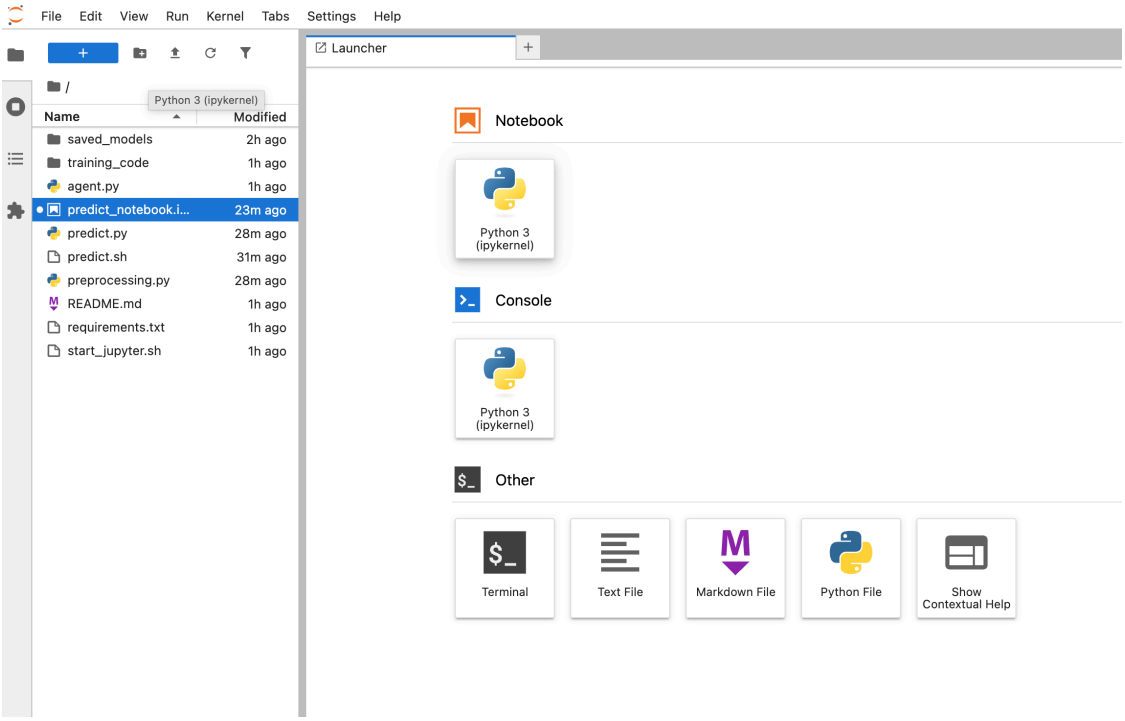
- Chạy start_jupyter.sh

> sudo docker run -it --gpus '"device=0"' -p 9777:9777 -v /mnt/zac2026/_data:/data:ro -v /mnt/zac2026/result:/result zac2026-teamname:v1 /bin/bash /code/start_jupyter.sh

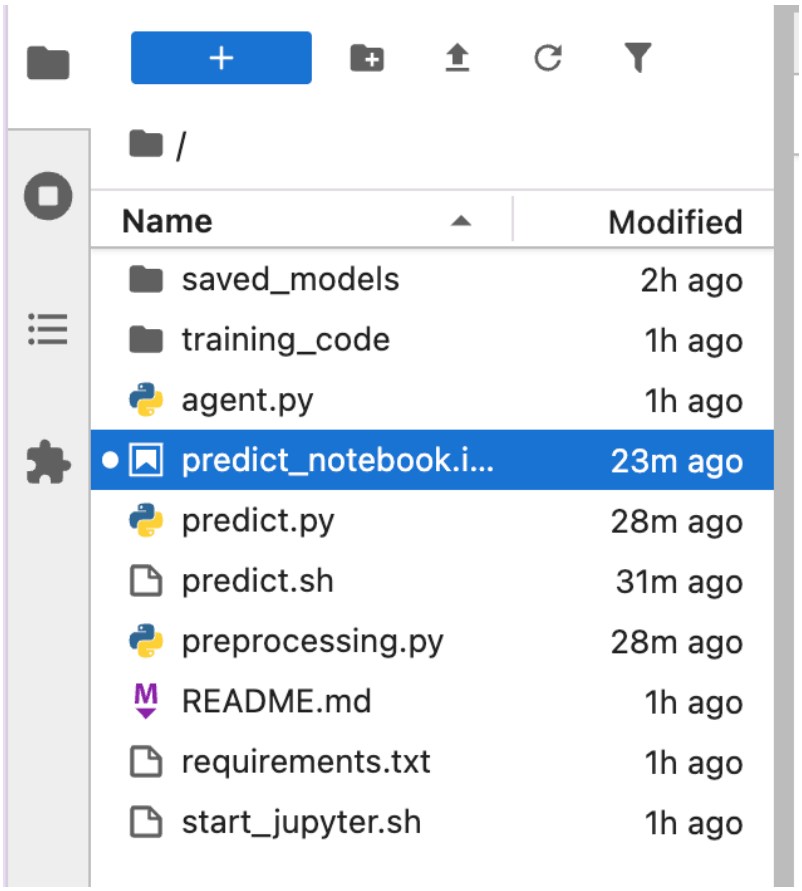
- Truy cập local host ở port 9777
- Nhập mật khẩu **zac2026**



- Giao diện cuối cùng sau khi cài đặt môi trường thành công



- Trong folder code phải có file predict_notebook.ipynb với nội dung như phần **phụ lục**



11. Xuất image ra và nộp bài.

> sudo docker save zac2026-teamname:v1 | gzip > zac2026_TeamName.tar.gz

File **zac2026_TeamName.tar.gz** là file bạn cần nộp. Kiểm tra lại trên máy khác: `docker load -i zac2026_TeamName.tar.gz` rồi chạy lại bước 10.1.

```
$ ls
agent.py                predict.py  preprocessing.py  requirements.txt  start_jupyter.sh  zac2026_BTC_test_v1.tar.gz
predict_notebook.ipynb  predict.sh  README.md         saved_models      training_code
```

6. Bổ sung bắt buộc về Training Code & Tài liệu mô tả

1. **README.md mô tả ý tưởng**

Các đội điền thêm **README.md** trình bày ngắn gọn ý tưởng của solution, gồm training và inference: model ASR, model LLM (tên, số tham số, context length), cách gọi tool / vòng lặp agent, dữ liệu và cách sinh dữ liệu tổng hợp (nếu có).

2. **Cung cấp đầy đủ Training Code & Data**

BTC sẽ sử dụng code và data trong thư mục **training_code/** để reproduce lại quá trình training nhằm kiểm tra tính nhất quán của kết quả.

Vì BTC sẽ chạy reproduce có internet, các đội có thể:

- Upload **training_data** và **models** lên HuggingFace hoặc dịch vụ tương tự
- Trong training_code, cần ghi rõ URL tải xuống.

Lưu ý: dữ liệu và mô hình trên các nền tảng không được thay đổi sau deadline nộp docker. **Base model** dùng trong quá trình training **không được đưa vào bên trong Docker** để tránh làm Docker quá nặng. (chỉ đưa model đã fine-tune / weights dùng lúc inference).

3. **Cố định seed**

Để bảo đảm reproducibility, các đội phải:

- Set seed cố định trong training và inference.
- Đảm bảo mô hình reproduce lại từ code/data cho ra kết quả giống mô hình dùng trong inference.

7. Phụ lục: Cấu trúc viết Jupyter notebook để đo lường thời gian

Bên cạnh các tài liệu hướng dẫn thực thi các mã nguồn huấn luyện và dự đoán. Mỗi team cần chuẩn bị một Jupyter notebook để thuận tiện cho việc đánh giá thời gian. BTC sẽ dùng notebook này để đo lường thời gian chạy của mỗi test case.

Nội dung của file notebook này sẽ tương tự như file **predict.py,** BTC sẽ tính điểm thời gian từ bước load file cho tới bước inference ra kết quả cho mỗi testcase.

Notebook này sẽ thể hiện các bước của việc dự đoán được thực hiện tách rời nhau như: (1) Set Seed; (2) Nạp mô hình và tài nguyên; (3) đọc dữ liệu test cases (từng test case 1) (4) thực hiện dự đoán. Do vậy notebook này phải gồm ít nhất 3 ô (cell) tương ứng với 3 bước chính:

- Bước 1: **set seed cố định (đội thi tự chọn seed, điền seed vào trong notebook)** và BTC chỉ thực hiện run lấy kết quả

```python
import os
import torch
import random
import numpy as np

def seed_everything(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

seed_everything(42) # Ví dụ cho seed bằng 42
```

- Bước 2: Nạp mô hình và các tài nguyên cần thiết

```python
# không dùng API ngoài, vì inference sẽ không có internet
asr_model = ...
model = ...
```

- Bước 3: Đọc nội dung các test case từ thư mục đầu vào

```python
import json, sys
from pathlib import Path

ROOT = Path("/data")
SPLIT = next(p.name for p in ROOT.iterdir() if (p / f"{p.nam
e}_tools_catalog.json").exists())
sys.path.insert(0, str(ROOT / SPLIT))   # zaic_tools của spli
t
from zaic_tools import ToolEnv
```

```
with open(ROOT / SPLIT / f"{SPLIT}.jsonl", encoding="utf-8")
as f:
    test_cases = [json.loads(line) for line in f]
```

- Bước 4: Thực hiện dự đoán và in ra kết quả cũng như thời gian.

```
import time

all_result = []
for item in test_cases:                                  # lần lượt
từng mẫu
    t1 = time.perf_counter()
    env = ToolEnv(SPLIT, item["id"], root=ROOT)    # đúng một
ToolEnv cho mỗi mẫu
    run_agent(item, env)                                 # ASR -> L
LM -> env.call(...) -> ...
    tool_calls, state_hash = env.get_tool_calls(), env.get_st
ate_hash()
    ms = int((time.perf_counter() - t1) * 1000)
    all_result.append({"id": item["id"], "tool_calls": tool_c
alls, "state_hash": state_hash, "time": ms})

Path("/result").mkdir(parents=True, exist_ok=True)
with open("/result/jupyter_submission.jsonl", "w", encoding
="utf-8") as f:
    for r in all_result:
        f.write(json.dumps(r, ensure_ascii=False) + "\n")
```

(Mã nguồn mang tính chất tham khảo)

Lưu ý: BTC chỉ mở file notebook và run all cells, nhiệm vụ của các bạn phải đảm bảo các cells khi chạy không bị lỗi ở bước nào, và cuối cùng phải xuất ra được files:

/result/jupyter_submission.jsonl, cùng định dạng file **submission.jsonl** (có trường `time`).

Để tránh bị overwrite kết quả khi chạy bước predict.py, đối với các kết quả được xuất ra khi chạy jupyter, các bạn phải thêm prefix là "**jupyter_**"

Các đội phải đảm bảo khi BTC chấm điểm thì điểm của file jupyter_submission.jsonl phải giống với submission.jsonl thì BTC mới dùng trường `time` của file này để đánh giá thời gian.

8. Upload docker

1. Lấy checksum MD5 của file docker **zac2026_TeamName.tar.gz** đã nén ở mục 5. Cấu trúc thư mục code

   ```
   md5sum zac2026_TeamName.tar.gz
   ```

   Tham khảo thêm cách check MD5 ở Windows và MacOS:

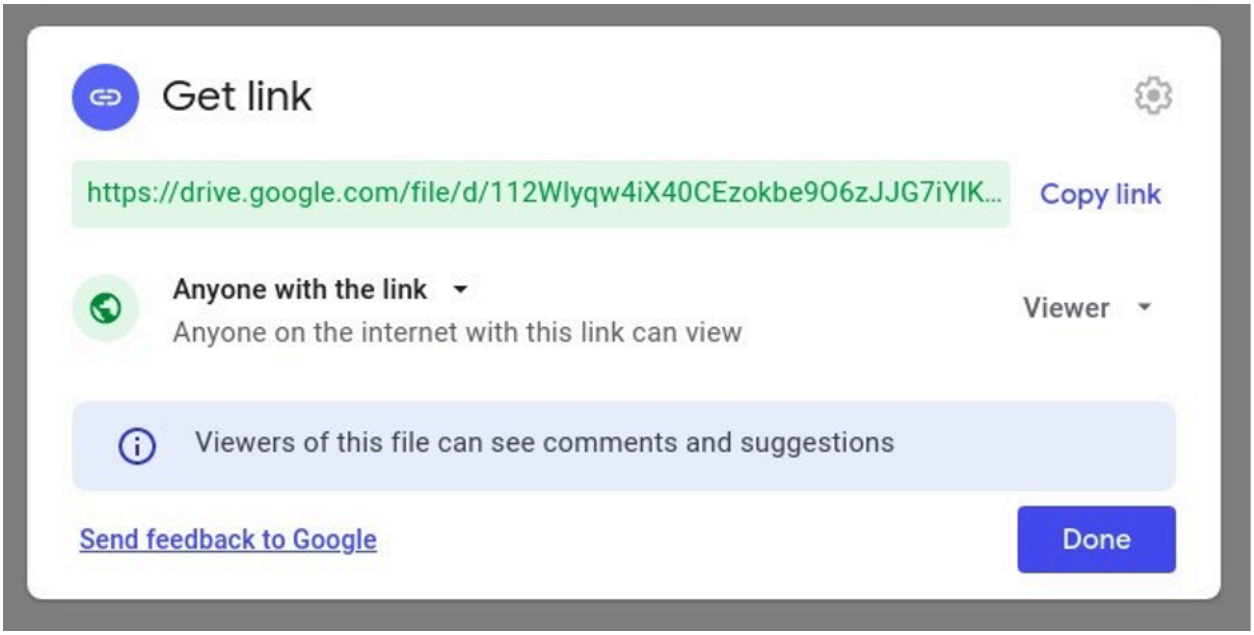
   https://portal.nutanix.com/page/documents/kbs/details?targetId=kA07V000000LWYqS

   Linux: https://www.geeksforgeeks.org/md5sum-linux-command/

2. Upload docker lên google drive

   BTC sẽ download docker của các bạn về máy chủ để tiến hành bước kiểm tra kết quả cuối cùng.

   Upload file docker đã nén lên dịch vụ storage của Google drive chỉnh sang chế độ share "Anyone with the link"



3. Nộp **link google drive** và **checksum** theo thông báo nộp bài của BTC

Chúc các bạn thành công!